We have an initial array of classic cars and we're interested in the total value of cars broken down by brand. We want to take our initial array, which contains six different cars from three different brands, and produce a new array with a total value for each of the three brands. We call `array_reduce()` with three arguments: the array we want to reduce, a reducer function (we're using a closure, but named functions are also fine) that will be called for every element, and an initial value.

The first time the closure is called, it is passed the initial value as the first argument (`$accumulator`) and the value of the first element. The reducer function *must* return the accumulator when it is complete. As the function is called for each element, `$accumulator` is modified. When the final element is processed, `array_reduce()` returns the final state of `$accumulator`. If there were no values to process (e.g., an empty array was supplied), `array_reduce()` would simply return the initial value.

In our example, a six-element multi-dimensional array of arrays, each with a sub-array containing three key/value pairs, has become a flat associative array containing three elements. Imagine if all we cared about was the total value of all cars: we could take our multi-dimensional `$stock` array and (with an appropriate reducer function) calculate a single integer. This is why the verb "reduce" is used to name the function.

Multi-dimensional Arrays and Recursive Functions

We have already seen multi-dimensional arrays in action earlier in the chapter, although the examples have been quite simple. Imagine you had a problem that involved processing something more complex, with three, four, or even more dimensions. Let's say our solution required that we examine every element in every array. Even a relatively simple question like "how many distinct occurrences are there of each value?" becomes difficult to answer, despite the fact that PHP has (you guessed it) a built-in function for counting the number of times a distinct value is found in an array:

```
$arr = ['Alice', 'Bob', 'Bob'];

$struct = [
    ['Skoda', 'Fabia', 'VRs'],
    ['Skoda', 'Fabia', 'VRs'],
];

print_r(array_count_values($arr));
print_r(array_count_values($struct));
```